2iE - Institut International d'Ingénierie de l'Eau et de l'Environnement

Master Analyse Spatiale avec Machine Learning

MODULE 2

ML Classique applique aux Données Spatiales

Dr Rodrique KAFANDO

Enseignant Chercheur, Informatique – Intelligence Artificielle

kafando.rodrique@gmail.com

Paramètre	Valeur
Programme	Master ASML — Semestre 1
Volume	CM : 4h | TD : 4h | TP : 6h
Prérequis	Module 2 — Feature matrix BF (ipc_phase + 9 features)
Outils	scikit-learn · XGBoost · SHAP · libpysal · GeoPandas · Folium
Données	M2_feature_matrix_BF.csv — 45 provinces × 9 features + ipc_phase
Fil conducteur	Prédire les phases IPC 1–5 par province et cartographier les résultats

Objectifs d'apprentissage

Au terme de ce module, vous serez en mesure de :

1. Expliquer le fonctionnement d'un Random Forest et d'un Gradient Boosting, et justifier le choix de l'un ou l'autre selon le contexte
2. Implémenter une spatial cross-validation pour éviter le data leakage géographique et produire des estimations de performance non biaisées
3. Évaluer un classifieur spatial avec des métriques adaptées : Cohen's Kappa, F1-macro, et Moran I des résidus
4. Interpréter les importances de features et identifier les variables les plus prédictives des phases IPC
5. Produire une carte choroplèthe Folium des phases IPC prédites, avec visualisation des erreurs
6. Comparer classification et régression ordinale pour une variable cible ordinale

1. Du Feature Engineering à la Prédiction

Le Module 2 a produit une feature matrix : un tableau de 45 lignes (provinces du Burkina Faso) et 9 colonnes (variables géospatiales) accompagné d'une variable cible — la phase IPC du Cadre Harmonisé. Ce tableau est exactement ce dont un algorithme de machine learning supervisé a besoin pour apprendre. Le Module 3 franchit l'étape suivante : entraîner des modèles, évaluer leurs performances correctement dans un contexte spatial, et transformer les prédictions en cartes utiles pour la décision humanitaire.

Mais cette transition du feature engineering à la prédiction cache un piège spécifique aux données géographiques. Un algorithme classique de ML suppose que les observations sont indépendantes — une province ne doit pas influencer les prédictions d'une autre. Or la dépendance spatiale mesurée en Module 2 (Moran I ≈ 0.3–0.5) viole systématiquement cette hypothèse. Ignorer ce problème conduit à des modèles qui semblent excellents en validation mais échouent en déploiement réel.

À retenir
Le Module 3 répond à trois questions dans l'ordre : (1) Comment prédire les phases IPC avec un bon modèle ? (2) Comment évaluer ce modèle sans se tromper sur ses performances réelles ? (3) Comment présenter et interpréter les résultats pour qu'ils soient utiles aux acteurs humanitaires ?

1.1 Rappel : la feature matrix M2

La feature matrix produite en M2 contient les variables suivantes. Chaque variable a été choisie pour son lien causal avec l'insécurité alimentaire, pas uniquement pour sa corrélation statistique.

Feature	Type	Lien causal avec IPC	VIF typique
superficie_km2	Géométrique	Province vaste → intervention difficile	2–4
compacite	Géométrique	Province allongée → accessibilité réduite	1–3
pop_density	Socio-éco	Pression sur les ressources alimentaires	3–6
fapar_mean	Spectral	Végétation actuelle → production agricole	2–4
fapar_trend	Spectral	Tendance dégradation terres	1–3
ipc_lag	Voisinage	Contagion régionale des crises	3–7
lisa_cat (encodé)	Voisinage	Appartenance à un cluster spatial	1–4
dist_route_km	Distance	Enclavement physique → accès aux marchés	2–5

Feature	Type	Lien causal avec IPC	VIF typique
travel_time_mean_h	Distance	Accessibilité fonctionnelle aux soins	3–6

Contexte terrain — Burkina Faso
Avec 45 observations (une par province), la feature matrix est petite pour du ML. C'est une réalité des données administratives africaines. Cette contrainte impose d'utiliser des modèles robustes face au surapprentissage (Random Forest avec profondeur limitée), une validation croisée adaptée (spatial CV), et une interprétation prudente des performances.

2. Random Forest : Principes et Application Spatiale

Le Random Forest est l'algorithme de référence pour les données tabulaires en machine learning. Il est particulièrement adapté à notre contexte : robuste face au sur-apprentissage sur les petits datasets, peu sensible aux hyperparamètres, capable de gérer les classes déséquilibrées, et il fournit directement les importances de features. Sa seule faiblesse majeure est l'extrapolation — il ne peut pas prédire au-delà de la plage des valeurs d'entraînement.

2.1 Comment fonctionne un Random Forest

Un Random Forest est un ensemble de B arbres de décision entraînés sur des sous-ensembles aléatoires des données (bootstrap) et des features (feature bagging). La prédiction finale est le vote majoritaire des B arbres (classification) ou leur moyenne (régression). La double randomisation — des données ET des features — garantit que les arbres sont diversifiés et que leurs erreurs ne sont pas corrélées.

Définition
Feature bagging : à chaque nœud de chaque arbre, le Random Forest ne considère que √p features (pour la classification) ou p/3 features (pour la régression), où p est le nombre total de features. Cela force les arbres à utiliser des features différentes et réduit leur corrélation, ce qui améliore la robustesse de l'ensemble.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
import pandas as pd
import numpy as np

# Chargement de la feature matrix produite en M2
df = pd.read_csv("M2_feature_matrix_BF.csv")

FEATURES = ["superficie_km2","compacite","pop_density",
            "fapar_mean","fapar_trend","ipc_lag",
            "dist_route_km","travel_time_mean_h"]
TARGET   = "ipc_phase"
```

```
X = df[FEATURES].values
y = df[TARGET].astype(int).values

# Random Forest — configuration initiale
rf = RandomForestClassifier(
    n_estimators=200,     # Nombre d'arbres : 200 est un bon défaut
    max_depth=5,          # Profondeur max : limiter le surapprentissage
    min_samples_leaf=2,   # Feuilles avec au moins 2 obs (robustesse)
    class_weight="balanced",  # Compenser le déséquilibre des classes IPC
    random_state=42,
    n_jobs=-1             # Utiliser tous les cœurs disponibles
)
```

2.2 Hyperparamètres critiques

Hyperparamètre	Valeur défaut	Rôle	Conseil BF (45 obs)
n_estimators	200–500	Nombre d'arbres	200 suffisent pour 45 observations
max_depth	None	Profondeur max des arbres	5–8 pour éviter le surapprentissage sur petit dataset
min_samples_leaf	1	Taille min d'une feuille terminale	2–3 : généralisation meilleure
max_features	sqrt(p)	Features considérées à chaque nœud	Ne pas changer pour la classification
class_weight	None	Poids des classes	balanced si classes déséquilibrées (IPC 4 rare)
bootstrap	True	Tirage avec remise pour chaque arbre	Toujours True pour RF standard

Attention
Avec seulement 45 observations, un Random Forest sans contrainte (max_depth=None) s'adaptera parfaitement aux données d'entraînement mais fera une mauvaise généralisation. Limitez max_depth à 5–8 et min_samples_leaf à 2–3 pour forcer le modèle à apprendre des règles généralisables plutôt qu'à mémoriser les données.

2.3 Gestion des classes déséquilibrées

Les phases IPC ne sont pas équidistribuées : les phases 3 et 2 sont les plus fréquentes, la phase 4 est rare, la phase 5 quasi absente au niveau provincial. Un classifieur naïf apprend à prédire "phase 2 ou 3" pour tout le monde et obtient une accuracy acceptable — mais rate

complètement les situations d'urgence. class_weight="balanced" corrige ce biais en donnant plus de poids aux classes minoritaires.

```
import collections
print("Distribution des phases IPC :")
print(collections.Counter(y))
# Exemple : {2: 14, 3: 16, 4: 9, 1: 4, 5: 2}
# Phase 5 sous-représentée → class_weight="balanced" indispensable

# Alternative : SMOTE (suréchantillonnage) pour les classes minoritaires
# from imblearn.over_sampling import SMOTE
# X_res, y_res = SMOTE(k_neighbors=2, random_state=42).fit_resample(X, y)
# SMOTE avec k_neighbors=2 car peu d'observations par classe
```

3. Gradient Boosting et XGBoost

Le Gradient Boosting est une alternative au Random Forest qui construit les arbres séquentiellement, chaque arbre corrigeant les erreurs du précédent. XGBoost (Extreme Gradient Boosting) en est l'implémentation la plus efficace et la plus utilisée en compétition de données. Sur des datasets tabulaires, XGBoost surpasse souvent le Random Forest — mais il est plus sensible aux hyperparamètres et peut surapprendre plus facilement, ce qui est particulièrement risqué avec 45 observations.

3.1 Random Forest vs Gradient Boosting : différences clés

Aspect	Random Forest	XGBoost
Construction	Arbres en parallèle (indépendants)	Arbres en séquence (chacun corrige le précédent)
Vitesse d'entraînement	Rapide (parallélisable)	Plus lent (séquentiel)
Surapprentissage	Robuste (moyenne de nombreux arbres)	Risqué si n_estimators trop grand
Hyperparamètres	Peu critiques	Nombreux et sensibles
Données manquantes	Nécessite imputation préalable	Gère nativement
Avec 45 obs	Recommandé — stable	Possible mais régularisation forte nécessaire
Features importantes	Gini importance	Gain importance + SHAP

Conseil pratique
Pour le dataset BF (45 observations), commencez toujours par le Random Forest. Il est plus robuste et plus facile à interpréter. Utilisez XGBoost comme comparaison avec une régularisation forte (max_depth=3, learning_rate=0.05, n_estimators=100). Ne choisissez XGBoost que s'il surpasse significativement RF en spatial CV.

3.2 Implémentation XGBoost

```
from xgboost import XGBClassifier
from sklearn.preprocessing import LabelEncoder

# XGBoost nécessite des labels de 0 à n_classes-1
le = LabelEncoder()
y_enc = le.fit_transform(y)   # IPC 1-5 → 0-4

xgb = XGBClassifier(
    n_estimators=100,        # Peu d'arbres pour éviter le surapprentissage
    max_depth=3,             # Arbres peu profonds
    learning_rate=0.05,      # Taux d'apprentissage faible → régularisation implicite
    subsample=0.8,           # 80% des données par arbre (comme RF bootstrap)
    colsample_bytree=0.8,    # 80% des features par arbre
    reg_alpha=0.1,           # Régularisation L1
    reg_lambda=1.0,          # Régularisation L2
    eval_metric="mlogloss",
    random_state=42,
    use_label_encoder=False,
    verbosity=0
)
```

4. Spatial Cross-Validation : Évaluer Honnêtement

La validation croisée est la technique standard pour estimer les performances d'un modèle ML sur des données non vues. Elle consiste à entraîner le modèle sur une partie des données et à le tester sur le reste, en répétant l'opération plusieurs fois. Mais la k-fold CV classique — qui divise les données aléatoirement — produit des estimations biaisées sur des données spatiales. Comprendre pourquoi et comment corriger ce biais est l'une des compétences les plus importantes de ce module.

4.1 Le problème du data leakage géographique

Supposons un dataset de 45 provinces. En k-fold classique avec k=5, chaque fold de test contient 9 provinces prises aléatoirement. Parmi ces 9 provinces, certaines sont géographiquement proches des provinces d'entraînement — et nous avons mesuré en Module 2 que les provinces proches ont des phases IPC similaires (Moran I ≈ 0.3–0.5). Le modèle "triche" : il apprend des patterns locaux très spécifiques aux clusters géographiques, et quand il prédit pour une province test, cette province est souvent proche d'une province d'entraînement qui lui ressemble.

Ce phénomène s'appelle le data leakage géographique. Il peut conduire à des estimations d'accuracy surestimées de 15 à 40 % selon le degré d'autocorrélation spatiale. Le modèle semble excellent en validation — et se révèle médiocre quand on l'applique à de nouvelles données.

Attention
Le data leakage géographique est insidieux car il ne produit aucun message d'erreur. Il faut activement le tester en calculant le Moran I des résidus : si les erreurs du modèle sont elles-mêmes spatialement autocorrélées (Moran I des résidus > 0 et significatif), c'est la preuve que le modèle a "appris" la structure spatiale sans la capturer dans ses features. La solution est soit d'ajouter des features de voisinage (fait en M2), soit d'utiliser une spatial CV.

4.2 Spatial Block Cross-Validation

La spatial block CV résout le problème en créant des folds géographiquement continus. Au lieu de diviser aléatoirement, on divise l'espace en blocs géographiques — des groupes de provinces contiguës. Quand un bloc entier devient le fold de test, les provinces test sont géographiquement séparées des provinces d'entraînement, ce qui empêche le leakage.

Définition
Spatial Block CV : les folds sont définis par la géographie, pas par le tirage aléatoire. Avec 45 provinces BF divisées en 5 blocs géographiques Nord/Centre-Nord/Centre/Sud-Ouest/Est, chaque fold de test contient des provinces non contiguës aux provinces d'entraînement. Les performances estimées sont représentatives des performances réelles sur de nouvelles zones.

```
import geopandas as gpd
from sklearn.model_selection import cross_val_score, KFold
from sklearn.metrics import cohen_kappa_score, f1_score
import numpy as np

# ——— Spatial Block CV — implémentation manuelle ———————————————————————

# Définition des blocs géographiques par région BF
BLOCS_GEO = {
    "Nord-Sahel":     ["Sahel","Nord","Centre-Nord"],    # Bloc 1
    "Centre-Ouest":   ["Centre","Centre-Ouest","Boucle du Mouhoun"], # Bloc 2
    "Centre-Est":     ["Centre-Est","Plateau Central","Centre-Sud"], # Bloc 3
    "Hauts-Bassins":  ["Hauts-Bassins","Cascades","Sud-Ouest"],     # Bloc 4
    "Est":            ["Est"],                          # Bloc 5
}

def spatial_block_cv(df, model, features, target, blocs):
    """Spatial block CV — entraîne sur 4 blocs, teste sur 1."""
    resultats = []
    for nom_test, regions_test in blocs.items():
```

```
    # Masque : provinces dans le bloc de test
    if "region" in df.columns:
        mask_test  = df["region"].isin(regions_test)
    else:
        # Fallback : découpage par latitude (centroid_lat)
        mask_test = df.index.isin(
            df.sample(frac=0.2, random_state=hash(nom_test)%100).index)
    mask_train = ~mask_test

    X_train = df.loc[mask_train, features].values
    X_test  = df.loc[mask_test,  features].values
    y_train = df.loc[mask_train, target].astype(int).values
    y_test  = df.loc[mask_test,  target].astype(int).values

    if len(X_test) == 0 or len(np.unique(y_train)) < 2:
        continue  # Bloc vide ou classes insuffisantes

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    kappa = cohen_kappa_score(y_test, y_pred) if len(y_test)>1 else 0
    f1    = f1_score(y_test, y_pred, average="macro",
                     zero_division=0)
    acc   = (y_test == y_pred).mean()
    resultats.append({"bloc":nom_test,"n_test":len(y_test),
                      "kappa":kappa,"f1_macro":f1,"accuracy":acc})

  return pd.DataFrame(resultats)

# Exécution
df_cv = spatial_block_cv(df, rf, FEATURES, TARGET, BLOCS_GEO)
print(df_cv.to_string(index=False))
print(f"\nKappa moyen   : {df_cv.kappa.mean():.3f} ± {df_cv.kappa.std():.3f}")
print(f"F1-macro moyen: {df_cv.f1_macro.mean():.3f} ± {df_cv.f1_macro.std():.3f}")
```

4.3 Comparaison CV classique vs Spatial CV

Pour mesurer l'amplitude du biais introduit par la CV classique, il est instructif de comparer les deux méthodes sur le même modèle.

```
from sklearn.model_selection import StratifiedKFold

# CV classique (biaisée pour données spatiales)
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores_classic = cross_val_score(rf, X, y,
```

```
                        cv=kf, scoring="accuracy")

# Résultats à comparer
print(f"CV classique  : {scores_classic.mean():.3f} ± {scores_classic.std():.3f}")
print(f"Spatial CV    : {df_cv.accuracy.mean():.3f} ± {df_cv.accuracy.std():.3f}")
print(f"Écart (biais) : {scores_classic.mean()-df_cv.accuracy.mean():+.3f}")
# Écart positif = CV classique surestimait les performances
```

À retenir

Un écart de +0.10 à +0.25 entre CV classique et spatial CV est typique sur des données IPC BF. Cela signifie que le modèle semblait avoir 70% d'accuracy en CV classique, mais n'atteint que 45–60% en estimation honnête. La spatial CV est la seule métrique à reporter dans une publication ou un rapport d'évaluation.

5. Métriques d'Évaluation Adaptées

L'accuracy — le pourcentage de prédictions correctes — est une métrique trompeuse quand les classes sont déséquilibrées. Si 40% des provinces sont en phase 3, un modèle qui prédit systématiquement "phase 3" obtient 40% d'accuracy sans avoir rien appris d'utile. Pour les phases IPC, trois métriques complémentaires sont recommandées.

5.1 Cohen's Kappa — la métrique de référence IPC

Définition

Cohen's Kappa : $\kappa = (po - pe) / (1 - pe)$

Où po = proportion de prédictions correctes observées, pe = proportion attendue par chance. $\kappa = 1$: accord parfait. $\kappa = 0$: accord dû au hasard uniquement. $\kappa < 0$: pire que le hasard. Pour des classifications de phases IPC, un $\kappa > 0.40$ est considéré "modéré à bon" selon l'échelle de Landis & Koch.

Kappa κ	Interprétation	Recommandation pour IPC BF
< 0.20	Faible — quasi-aléatoire	Modèle non utilisable
0.20–0.40	Modéré	Acceptable pour analyse exploratoire
0.40–0.60	Bon	Utilisable pour aide à la décision
0.60–0.80	Très bon	Fiable pour priorisation humanitaire
> 0.80	Excellent	Très difficile à atteindre sur données IPC réelles

5.2 F1-macro et matrice de confusion

Le F1-macro calcule le F1-score pour chaque classe séparément puis fait la moyenne, en accordant le même poids à toutes les classes. C'est crucial pour les phases IPC : prédire correctement les rares phases 4 et 5 (urgence et famine) est plus important que prédire les phases 2 et 3, qui sont les plus fréquentes.

```
from sklearn.metrics import (classification_report, confusion_matrix,
                             ConfusionMatrixDisplay, cohen_kappa_score)
import matplotlib.pyplot as plt

# Entraînement final sur toutes les données pour diagnostics
rf.fit(X, y)
y_pred_full = rf.predict(X)

# Rapport complet
print(classification_report(y, y_pred_full,
    target_names=[f"IPC {i}" for i in sorted(np.unique(y))]))

# Matrice de confusion
cm = confusion_matrix(y, y_pred_full, labels=sorted(np.unique(y)))
fig, ax = plt.subplots(figsize=(7, 6))
disp = ConfusionMatrixDisplay(cm,
        display_labels=[f"IPC {i}" for i in sorted(np.unique(y))])
disp.plot(ax=ax, colorbar=False, cmap="Blues")
ax.set_title("Matrice de confusion — Phases IPC BF\n(RF entraîné sur toutes les données)",
         fontsize=11, fontweight="bold")
plt.tight_layout()
plt.savefig("M3_confusion_matrix.png", dpi=150)
```

5.3 Moran I des résidus — test du leakage résiduel

Même après la spatial CV, il est utile de vérifier si les erreurs du modèle sont spatialement aléatoires. Si les provinces voisines ont tendance à être toutes mal prédites ensemble, cela indique que le modèle n'a pas capturé toute la structure spatiale disponible dans les features.

```
from esda.moran import Moran
from libpysal.weights import Queen

# Résidus = différence entre prédiction et réalité
# Pour classification : 0 si correct, valeur non nulle sinon
gdf["y_pred"] = rf.predict(X)
gdf["residu"] = (gdf["y_pred"] - gdf["ipc_phase"]).abs()

# Test Moran I sur les résidus
W = Queen.from_dataframe(gdf, use_index=True)
W.transform = "r"
moran_res = Moran(gdf["residu"].fillna(0).values, W)
print(f"Moran I résidus = {moran_res.I:.4f} (p = {moran_res.p_sim:.4f})")
if moran_res.p_sim < 0.05 and moran_res.I > 0.10:
```

```
    print("⚠️ Autocorrélation résiduelle significative")
    print("→ Ajouter des features de voisinage manquantes")
    print("→ Ou utiliser un modèle avec kernel spatial (SVM RBF)")
else:
    print("✅ Résidus spatialement aléatoires — modèle satisfaisant")
```

6. Interprétabilité : Feature Importance et SHAP

Un modèle qui prédit sans expliquer est peu utile pour la décision humanitaire. Les responsables du CILSS et de la DGRE ont besoin de savoir POURQUOI telle province est prédite en phase 4, pas seulement qu'elle l'est. L'interprétabilité est aussi cruciale pour la validation du modèle : si les features les plus importantes sont biologiquement absurdes (par exemple : fapar_trend moins important que superficie_km2), cela doit alerter sur un problème de données ou d'architecture.

6.1 Feature Importance Gini

Le Random Forest calcule automatiquement l'importance de chaque feature en mesurant la réduction moyenne d'impureté (Gini) que cette feature apporte aux nœuds de séparation. C'est une mesure relative : les importances somment à 1. Elle est rapide à calculer mais biaisée en faveur des features à haute cardinalité et des features corrélées.

```
import matplotlib.pyplot as plt
import pandas as pd

# Feature importance Gini (built-in RF)
importances = pd.Series(rf.feature_importances_, index=FEATURES)
importances = importances.sort_values(ascending=True)

fig, ax = plt.subplots(figsize=(9, 5))
bars = ax.barh(importances.index, importances.values,
               color="steelblue", edgecolor="white")
ax.set_xlabel("Importance (Gini mean decrease in impurity)", fontsize=10)
ax.set_title("Feature Importance — Random Forest IPC BF\n"
             "(plus élevée = plus prédictive des phases alimentaires)",
             fontsize=11, fontweight="bold")
ax.axvline(1/len(FEATURES), color="red", linestyle="--", alpha=0.5,
           label=f"Uniforme ({1/len(FEATURES):.2f})")
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig("M3_feature_importance.png", dpi=150)
plt.show()
```

6.2 Permutation Importance — plus fiable

La permutation importance est plus robuste que l'importance Gini. Elle mesure combien les performances du modèle se dégradent quand on mélange aléatoirement les valeurs d'une feature — ce qui rompt le lien entre la feature et la cible. Si les performances chutent beaucoup, la feature est vraiment importante.

```
from sklearn.inspection import permutation_importance

perm_imp = permutation_importance(
    rf, X, y,
    n_repeats=50,        # 50 permutations pour stabiliser
    scoring="balanced_accuracy",
    random_state=42
)

perm_df = pd.DataFrame({
    "Feature": FEATURES,
    "Importance": perm_imp.importances_mean,
    "Std": perm_imp.importances_std
}).sort_values("Importance", ascending=False)

print("=== Permutation Importance (plus fiable que Gini) ===")
print(perm_df.to_string(index=False))
print("\n→ Comparer avec Gini : même classement ?")
print("→ ipc_lag doit être dans le top-3 (justifie le travail M2).")
```

Contexte terrain — Burkina Faso
Résultat attendu sur données IPC BF : ipc_lag devrait être dans le top-2 des features les plus importantes, avec fapar_mean. Cela valide rétrospectivement la construction de la matrice W en M2 et le calcul du lag spatial. Si ipc_lag n'est pas dans le top-3, cela peut indiquer que le Moran I des données est trop faible pour que le voisinage soit informative.

6.3 SHAP — aperçu (approfondi en Module 6)

SHAP (SHapley Additive exPlanations) est la méthode d'interprétabilité la plus rigoureuse mathématiquement. Elle décompose chaque prédiction en contributions individuelles de chaque feature, basées sur la théorie des jeux coopératifs. Une valeur SHAP positive pour ipc_lag signifie que cette feature pousse la prédiction vers une phase plus élevée pour cette province spécifique.

```
import shap

# Explainer TreeSHAP — optimisé pour les arbres de décision
explainer = shap.TreeExplainer(rf)
shap_values = explainer.shap_values(X)
# shap_values est une liste de n_classes tableaux (n_obs × n_features)
```

```
# Beeswarm plot — vue globale de toutes les features et observations
shap.summary_plot(shap_values, X, feature_names=FEATURES,
                  class_names=[f"IPC {i}" for i in sorted(np.unique(y))],
                  plot_type="bar", show=False)
plt.tight_layout()
plt.savefig("M3_shap_summary.png", dpi=150)
plt.show()

# Waterfall plot pour une province spécifique (ex. province la plus à risque)
idx_pire = np.argmax(y)   # Province avec le plus haut IPC
shap.waterfall_plot(shap.Explanation(
    values=shap_values[y[idx_pire]-1][idx_pire],
    base_values=explainer.expected_value[y[idx_pire]-1],
    data=X[idx_pire], feature_names=FEATURES)
```

7. Carte Prédictive des Phases IPC

La prédiction d'un modèle ML n'est utile que si elle est présentée de façon compréhensible par les acteurs humanitaires. Une carte choroplèthe des phases IPC prédites, comparée aux phases observées du Cadre Harmonisé, est le livrable final de ce module. Elle permet d'identifier les provinces où le modèle est fiable, celles où il diverge du terrain, et les zones à surveiller en priorité.

7.1 Prédiction et carte Folium

```
# Entraînement final sur TOUTES les données
rf.fit(X, y)
gdf["ipc_pred"] = rf.predict(X)
gdf["ipc_proba_max"] = rf.predict_proba(X).max(axis=1)
gdf["erreur"] = (gdf["ipc_pred"] != gdf["ipc_phase"]).astype(int)
gdf["ecart"]  = abs(gdf["ipc_pred"] - gdf["ipc_phase"])

# Carte Folium — 3 couches : observé, prédit, erreurs
m = folium.Map(location=[12.37, -1.54], zoom_start=6,
               tiles="CartoDB positron")

PALETTE_IPC = {1:"#27ae60",2:"#f1c40f",3:"#e67e22",4:"#e74c3c",5:"#8e44ad"}

# Couche 1 : phases prédites
fg_pred = folium.FeatureGroup(name="Phases IPC prédites (RF)")
for _, row in gdf.iterrows():
    pred  = int(row["ipc_pred"])
    obs   = int(row["ipc_phase"]) if pd.notna(row["ipc_phase"]) else "?"
    proba = float(row["ipc_proba_max"])
```

```
    nom   = str(row.get("province", row.iloc[0]))[:15]
    popup_html = (
        "<b>" + nom + "</b><br>"
        + "IPC prédit : " + str(pred) + "<br>"
        + "IPC observé : " + str(obs) + "<br>"
        + "Confiance : " + f"{proba:.0%}"
    )
    folium.GeoJson(
        row.geometry.__geo_interface__,
        style_function=lambda f, c=PALETTE_IPC.get(pred,"#95a5a6"): {
            "fillColor":c,"color":"white","weight":0.5,"fillOpacity":0.7},
        popup=folium.Popup(popup_html, max_width=200),
        tooltip=nom
    ).add_to(fg_pred)
fg_pred.add_to(m)

# Couche 2 : erreurs (provinces mal prédites)
fg_err = folium.FeatureGroup(name="Erreurs de prédiction")
for _, row in gdf[gdf["erreur"]==1].iterrows():
    nom = str(row.get("province", row.iloc[0]))[:15]
    folium.GeoJson(
        row.geometry.__geo_interface__,
        style_function=lambda f: {
            "fillColor":"none","color":"#e74c3c","weight":3,"fillOpacity":0},
        tooltip=f"{nom} — Prédit:{int(row.ipc_pred)} Observé:{int(row.ipc_phase)}"
    ).add_to(fg_err)
fg_err.add_to(m)

folium.LayerControl().add_to(m)
m.save("M3_carte_predictions_IPC.html")
print("Carte interactive sauvegardée : M3_carte_predictions_IPC.html")
```

7.2 Carte comparative statique

```
fig, axes = plt.subplots(1, 3, figsize=(20, 7))

palette = {1:"#27ae60",2:"#f1c40f",3:"#e67e22",4:"#e74c3c",5:"#8e44ad"}

# Observé
gdf.plot(color=gdf["ipc_phase"].map(palette),
         edgecolor="white", linewidth=0.5, ax=axes[0])
axes[0].set_title("IPC observé\n(Cadre Harmonisé)", fontweight="bold")
axes[0].axis("off")
```

```
# Prédit
gdf.plot(color=gdf["ipc_pred"].map(palette),
        edgecolor="white", linewidth=0.5, ax=axes[1])
axes[1].set_title("IPC prédit\n(Random Forest)", fontweight="bold")
axes[1].axis("off")

# Écart
gdf.plot(column="ecart", cmap="RdYlGn_r", legend=True, vmin=0, vmax=2,
        legend_kwds={"label":"Écart |prédit - observé|","orientation":"vertical"},
        edgecolor="white", linewidth=0.5, ax=axes[2])
axes[2].set_title("Écart de prédiction\n(0=correct, 2=erreur de 2 phases)",
fontweight="bold")
axes[2].axis("off")

import matplotlib.patches as mpatches
leg = [mpatches.Patch(color=v,label=f"Phase {k}") for k,v in palette.items()]
axes[0].legend(handles=leg, loc="lower left", fontsize=8)
axes[1].legend(handles=leg, loc="lower left", fontsize=8)

plt.suptitle("Random Forest IPC — Observé vs Prédit\n"
           f"Spatial CV Kappa = {df_cv.kappa.mean():.3f}",
           fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("M3_carte_comparative.png", dpi=150, bbox_inches="tight")
plt.show()
```

8. Classification vs Régression Ordinale

Les phases IPC sont une variable ordinale : la phase 3 est "entre" la phase 2 et la phase 4, et un classifieur multiclasses standard ne tient pas compte de cet ordre. Prédire "phase 2" quand la réalité est "phase 4" est une erreur plus grave que prédire "phase 3". La régression ordinale — ou plus simplement le Random Forest Regressor — peut respecter cet ordre et produire de meilleures métriques.

8.1 Random Forest Regressor pour la prédiction ordinale

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score

rf_reg = RandomForestRegressor(
    n_estimators=200,
    max_depth=6,
    min_samples_leaf=2,
    random_state=42,
```

```
    n_jobs=-1
)

# Prédictions et arrondi à la phase la plus proche
rf_reg.fit(X, y.astype(float))
y_pred_reg  = rf_reg.predict(X)
y_pred_int  = np.clip(np.round(y_pred_reg).astype(int), 1, 5)

mae = mean_absolute_error(y, y_pred_reg)
print(f"MAE = {mae:.3f} phases  (erreur moyenne en phases IPC)")
print(f"R²  = {r2_score(y, y_pred_reg):.3f}")
print(f"Kappa (arrondi) = {cohen_kappa_score(y, y_pred_int):.3f}")
```

8.2 Tableau comparatif des approches

Approche	Avantage	Inconvénient	Recommandation BF
RF Classifieur	Multiclasse natif, probabilités par phase	Ignore l'ordre des phases	Baseline — toujours tester en premier
RF Régresseur	Respecte l'ordre ordinal, MAE interprétable	Arrondi peut introduire des artefacts	Tester si RF classifieur κ < 0.40
XGBoost Classifieur	Souvent plus précis que RF	Risque surapprentissage, +sensible HP	Comparer avec RF en spatial CV
Régression logistique ordinale	Interprétation statistique claire	Suppose linéarité — souvent trop simple	Baseline théorique — rarement optimal

À retenir
Ne choisissez pas le modèle avec la meilleure accuracy en k-fold classique — choisissez celui avec le meilleur Kappa en spatial CV. Ce sont souvent des modèles différents. Reportez systématiquement la méthode de validation utilisée dans tout rapport ou publication.

9. Bonnes Pratiques et Erreurs Fréquentes

#	Erreur	Conséquence	Correction
1	Utiliser un k-fold classique sur données spatiales	Performance surestimée de 15–40%	Spatial block CV obligatoire
2	Ignorer le déséquilibre des classes IPC	Modèle prédit toujours la classe majoritaire	class_weight="balanced"
3	Reporter accuracy sans Kappa	Métrique trompeuse sur classes déséquilibrées	Toujours reporter Kappa + F1-macro
4	Ne pas tester Moran I des résidus	Data leakage résiduel non détecté	Calculer Moran I des résidus systématiquement
5	Oublier LabelEncoder pour XGBoost	ValueError : classes doivent partir de 0	LabelEncoder() avant XGBClassifier
6	Conclure sur données d'entraînement seules	Surapprentissage non détecté	Toujours évaluer sur données non vues (spatial CV)
7	Trop d'hyperparamètres sur 45 observations	Surapprentissage quasi-certain	Garder les defaults de RF, régulariser fortement XGBoost

9.1 Vers le Module 4 — Deep Learning Spatial

Ce module a couvert les algorithmes de ML classique — Random Forest et XGBoost — appliqués à des features tabulaires géospatiales. Ces modèles sont performants mais ne peuvent pas traiter directement des images satellites ou des séries temporelles spatiales. Le Module 4 introduit les réseaux de neurones convolutifs (CNN) pour les données raster et les LSTM pour les séries temporelles, puis Google Earth Engine pour l'accès à des pétaoctets d'archives satellitaires.

À retenir
Un bon modèle ML classique sur une bonne feature matrix bat presque toujours un réseau de neurones mal configuré sur des données brutes. Ne passez au Deep Learning (M4) qu'après avoir établi un solide baseline en ML classique (M3). Cette baseline sera votre référence pour mesurer le gain réel apporté par les architectures plus complexes.

Références

- Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.

- Pohjankukka, J. et al. (2017). Estimating the prediction performance of spatial models via spatial k-fold cross-validation. IJGIS, 31(10).
- Ploton, P. et al. (2020). Spatial validation reveals poor predictive performance of large-scale ecological mapping models. Nature Communications, 11, 4540.
- Lundberg, S., Lee, S.-I. (2017). A unified approach to interpreting model predictions. NeurIPS 2017.
- Roberts, D. et al. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. Ecography, 40(8).
- scikit-learn : scikit-learn.org | XGBoost : xgboost.readthedocs.io | SHAP : shap.readthedocs.io